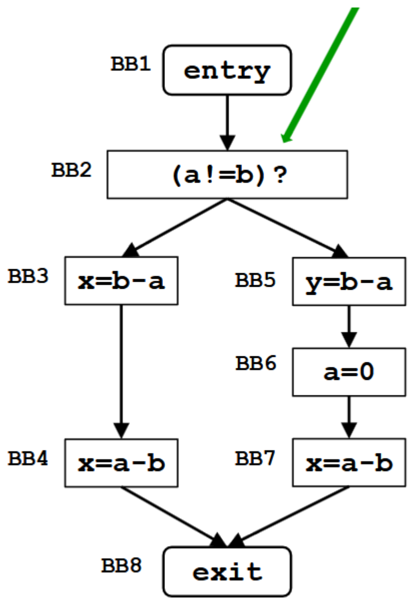
Compilers Assignment 2

Stefano Casolari 187572

July 14, 2026

Contents

1 Very Busy Expression



1.1 Definition

An expression is considered to be **very busy** in a specific instance p when said expression is evaluated on all path from p to the exit node without having its operands redefined along the way.

1.2 Data Flow Analysis

Backwards flow analysis proves efficient here since the expression must be evaluated on all paths to the exit. The meet operator is the intersection for the same reason

Domain	Set of all expressions
Direction	Backwards $In(B) = f(Out(B))$ $Out(B) = \wedge In(Succ(B))$
Transfer function	$In(B) = Use(B) U (Out(B) - Def(B))$
Meet operation	$\cap$
Boundary Condition	$In(Exit) = \emptyset$
Initial Interior Points	$In(B) = U$

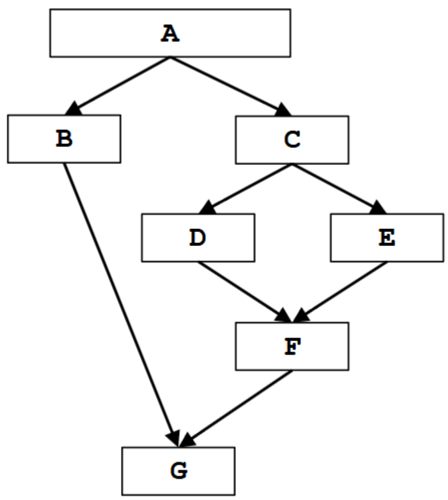
1.3 Example

Since there aren't any loop in the example cfg a single iteration is sufficent

	1:In(B)	1:Out(B)
BB1 (entry)	$b - a$	$b - a$
BB2	$b - a$	$b - a$
BB3	$a - b, b - a$	$a - b$
BB4	$a - b$	$\emptyset$
BB5	$b - a$	$\emptyset$
BB6	$\emptyset$	$a - b$
BB7	$a - b$	$\emptyset$
BB8 (exit)	$\emptyset$	$\emptyset$

Table 1: Very Busy Expression example table

2 Dominator Analysis



2.1 Definition

We say that a node B dominates a node N if all paths from the entry to node N pass through B. By this definition we can say that all blocks trivially dominate themselves.

2.2 Data Flow Analysis

Domain	Set of all nodes of the Graph
Direction	Forward $Out(B) = f(In(B))$ $In(B) = \wedge Out(Prev(B))$
Transfer function	$Out(B) = Gen(B)U(In(B) - Kill(B))$
Meet operation	$\cap$
Boundary Condition	$Out(Entry) = \{Entry\}$
Initial Interior Points	$Out(B) = U$

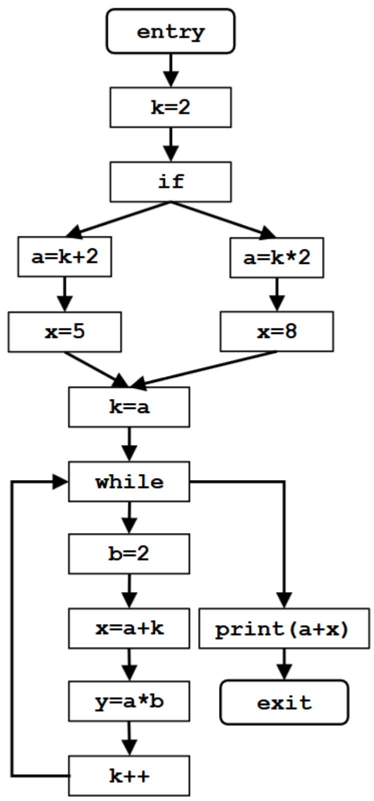
2.3 Example

Since there aren't any loop in the example cfg a single iteration is sufficent

	1:In(B)	1:Out(B)
A (entry)	$\emptyset$	A
B	A	A, B
C	A	A, C
D	A, C	A, C, D
E	A, C	A, C, E
F	A, C	A, C, F
G (exit)	A	A, G

Table 2: Dominator Analysis example table

3 Constant Propagation



3.1 Description

Constant Propagation evaluates variables and expression in a given point of the cfg as rappresentation of a known value, a constant. This evaluation extends beyond the simple act of assigning a value to a variable, as that known variable can be used to evaluate others as constant (this is the propagation part).

3.2 Data Flow Analysis

Constant propagation can be described as a forward Data flow analysis. the meet operator is the intersection operator (since a variable has to be constant on all path to the node)

Note that we cannot use a simply array of bits to rappresent the values since at a given time a variable can be unknown. The easiest way to implement this is having as a domain a set of ¡variable, value¿ elements, where a variable can only be in one element. if a variable is not present in our set then its value is unknown.

Domain	$\{(v, c)\}$:v variable of our cfg, c constant value
Direction	Forward $Out(B) = f(In(B))$ $In(B) = \wedge Out(Prev(B))$
Transfer function	$Out(B) = Gen(B)U(In(B) - Kill(B))$
Meet operation	$\cap$
Boundary Condition	$Out(Entry) = \emptyset$
Initial Interior Points	$Out(B) = U$

3.3 Example

the result are the same from the second iteration onwards

	1:In(B)	1:Out(B)	2:In(B)	2:Out(B)
$BB1$ (entry)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
$BB2$ k=2	$\emptyset$	$< k, 2 >$	$\emptyset$	$< k, 2 >$
$BB3$ if	$< k, 2 >$	$< k, 2 >$	$< k, 2 >$	$< k, 2 >$
$BB4$ a=k+2	$< k, 2 >$	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$	$< k, 2 >$ $< a, 4 >$
$BB5$ x=5	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$ $< a, 4 >$ $< x, 5 >$	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$ $< a, 4 >$ $< x, 5 >$
$BB6$ a=k*2	$< k, 2 >$	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$	$< k, 2 >$ $< a, 4 >$
$BB7$ x=8	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$ $< a, 4 >$ $< x, 8 >$	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$ $< a, 4 >$ $< x, 8 >$
$BB8$ k=a	$< k, 2 >$ $< a, 4 >$	$< k, 4 >$ $< a, 4 >$	$< k, 2 >$ $< a, 4 >$	$< k, 4 >$ $< a, 4 >$
$BB9$ while	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$ $< a, 4 >$	$< a, 4 >$	$< a, 4 >$
$BB10$ b=2	$< k, 2 >$ $< a, 4 >$	$< k, 2 >$ $< a, 4 >$ $< b, 2 >$	$< a, 4 >$	$< a, 4 >$ $< b, 2 >$
$BB11$ x=a+k	$< k, 2 >$ $< a, 4 >$ $< b, 2 >$	$< k, 2 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$	$< a, 4 >$ $< b, 2 >$	$< a, 4 >$ $< b, 2 >$
$BB12$ y=a*b	$< k, 2 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$	$< k, 2 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$ $< y, 8 >$	$< a, 4 >$ $< b, 2 >$	$< a, 4 >$ $< b, 2 >$ $< y, 8 >$
$BB13$ k++	$< k, 2 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$ $< y, 8 >$	$< k, 3 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$ $< y, 8 >$	$< a, 4 >$ $< b, 2 >$ $< y, 8 >$	$< a, 4 >$ $< b, 2 >$ $< y, 8 >$
$BB14$ print(a+x)	$< k, 3 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$ $< y, 8 >$	$< k, 3 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$ $< y, 8 >$	$< a, 4 >$ $< b, 2 >$ $< y, 8 >$	$< a, 4 >$ $< b, 2 >$ $< y, 8 >?$
$BB15$ exit	$< k, 3 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$ $< y, 8 >$	$< k, 3 >$ $< a, 4 >$ $< b, 2 >$ $< x, 6 >$ $< y, 8 >$	$< a, 4 >$ $< b, 2 >$ $< y, 8 >$	$< a, 4 >$ $< b, 2 >$ $< y, 8 >$

Table 3: Constant Propagation example table